

LEAN_SCOPE

Helios — Lean Scope (2 weeks · 1 developer · Claude Pro)

LEAN_SCOPE.md · v1.0 · 2026-06-03 · *The ruthless cut. What to build, what to kill.*

The thesis (read once, then act on it)

The red-team review killed the **apparatus** and the **overclaims** — not the **spine**. The genuinely valuable, honest, *achievable* core of Helios is four things:

1. A **governed dbt/BigQuery model** of the real GA4 funnel (sessionization, monotonic funnel, revenue).
2. The **mix-vs-rate decomposition** — the one technique that makes you look senior (it dodges the Simpson's-paradox mistake real analysts make).
3. **One MCP server + one grounded LLM step** — enough to truthfully claim "AI engineering," not a 7-agent fleet.
4. An **honest evaluation** that beats a naive baseline — and the maturity to say what it does *not* prove.

Everything else in the full design — the 7 agents, the 5 servers, the scheduler, the memory/vector store, the 47 metrics, the 50-scenario circular benchmark, the word *autonomous*, the phrase *diagnose why* — is **cut**. It is either theater on a frozen dataset, over-engineering for a batch job, or an overclaim that gets you destroyed in an interview.

Reframe the project (do this first). Not "Autonomous Growth Diagnosis Engine that diagnoses *why*." → "**Helios — a governed, mix-vs-rate funnel-diagnosis engine for GA4.**" It diagnoses *where* a metric moved and *how much it's worth*, with grounded SQL and an honest accuracy benchmark. This single rename neutralizes the three biggest red-team kills and makes everything below defensible.

Constraints, honestly read

- **2 weeks, 1 dev** → you can build the M0–M5 spine + a thin slice of M6/M7/M10. You cannot build M8–M12. The full design is 6–10 weeks; don't pretend otherwise.
- **Claude Pro is your *pair*, not an *API fleet*.** Claude Pro = Claude Code (your dev accelerator) + a modest usage budget. It is **not** an API budget to run 7 agents × many BigQuery round-trips per run. So the architecture must make **one** LLM call per diagnosis (the brief), maybe a second (a self-check). This constraint *forces* the right (lean) design.
- **The code already exists in the repo docs.** `DBT_GUIDE.md` has the full SQL; `MCP_ARCHITECTURE.md` has the server skeletons; `semantic_layer.yaml` and the scenarios exist. Your 2 weeks is **wiring + testing + the brief + the eval**, not design. That's what makes this feasible.

What carries the value (therefore: keep)

Value	Carried by	Keep
Business	A correct, governed, mix-vs-rate diagnosis + a dollar figure on real GA4 data	dbt marts + <code>decompose_change</code> + the brief
Resume	dbt · BigQuery · GA4 · semantic layer · MCP · grounded LLM · evaluation	the spine + 1 MCP server + 1 LLM step + honest eval
Interview	The decomposition (Simpson's), the grounding boundary, the <i>honest</i> eval, and the scoping decision itself	all of the above + this document

CUT LIST (be ruthless — here's the kill order and why)

Cut entirely (do not build, do not mention as "built"):

- **The 7-agent FSM.** → Replace with **one** grounded LLM call that narrates the deterministic decomposition. *Why: red-team #6 — apparatus disproportionate; and Claude Pro can't fund a fleet.*
- **4 of the 5 MCP servers** (warehouse, experiment, report). → Keep **one** (semantic or stats) so "I built an MCP server" is true; everything else is a plain Python function. *Why: over-abstraction for one dev.*
- **Autonomy / scheduler / cron / "always-on."** → Run it as a command. *Why: red-team #4 — theater on a frozen dataset.*
- **Memory / vector store / suppression / action-tracking.** *Why: over-engineering for a batch job on 3 months of frozen data.*

- **Cohorts / day-30 retention / RFM / forecasting.** *Why: the 3-month window can't support them; not core.*
- **experiment-mcp / power analysis / experiment *design*.** → A single "recommended next test" sentence in the brief. *Why: you can't run experiments on observational data anyway.*
- **The 50-scenario benchmark + CI gate.** → 6–10 honest scenarios, run locally. *Why: 50 is volume theater; the eval value is in honesty + a baseline, not count.*
- **CAC proxy, product/channel-revenue, ~35 of the 47 metrics.** → ~12 metrics. *Why: no cost data; bloat.*
- **The words "autonomous" and "diagnose why."** *Why: red-team #3/#4 — unsupported by the engine and the data.*

Defer to v2 (cut for *time*, not because they're flawed): the other agents, the other servers, the full benchmark + CI, the richer semantic layer, a real self-critique loop.

Keep (the spine): dbt staging → sessionization → funnel flags →

`fct_funnel` / `fct_daily_funnel` / `fct_orders` + ~4 dims; ~12 governed metrics;

`decompose_change` + significance; one MCP server; one grounded LLM brief; a small honest eval.

Helios MVP — end of Week 1

"Governed mix-vs-rate funnel diagnosis on real GA4 data." No LLM yet. Pure, correct, tested analytics engineering + the decomposition.

Scope / build (trimmed M0–M5 + the decomposition):

- M0–M2: repo, GCP/BigQuery, dbt config, macros, sources, staging (`stg_ga4__events` , `stg_ga4__event_params`).
- M3 (keystone): `int_ga4__sessionized` + `int_ga4__funnel_steps` (monotonic `reached_*`) — **golden tests first**.
- M4: `fct_funnel` , `fct_daily_funnel` , `fct_orders` + `dim_date` , `dim_channels` (+ `dim_users` , `dim_items` if time). Tests: monotonicity, `revenue_reconciles` to the cent, key uniqueness.
- M5 (trim): a **~12-metric** semantic YAML (sessions, the 4 funnel counts, the 4 conversion rates, `revenue` , `aov` , `revenue_per_session`).
- `decompose_change(metric, dim, t0, t1)` in plain Python (~15 lines) + a two-proportion significance test, with a **golden unit test** (the worked `mix=-0.0018` , `rate=0` example).
- `diagnose.py` : find the largest WoW movement in `session_conversion_rate` , decompose it by `device_category` and `channel_group` , attach a revenue figure, print a **templated** brief (no LLM).

Deliverable / demo: `python -m helios.diagnose` → a correct, governed mix-vs-rate diagnosis of the actual Google Merchandise Store funnel, with a dollar figure and a worked Simpson's-paradox case. + README.

The line it earns: *"Built a governed dbt/BigQuery model of a GA4 e-commerce funnel and a mix-vs-rate decomposition engine that distinguishes traffic-mix shifts from real conversion changes — the Simpson's-paradox trap — and prices the movement in dollars."* (100% true, senior-grade, no AI hype needed.)

Helios v1 — end of Week 2 (THIS is what you ship)

"+ one MCP server + one grounded LLM brief + an honest evaluation." This is the resume/interview centerpiece.

Scope / build (thin M6 + M7 + small M10):

- **One MCP server** — `semantic-mcp` (`build_query` from the YAML) or `stats-mcp` (`decompose_change` / `significance`). Connect it to **Claude Code** (your existing Pro access acts as the MCP client). Now the claim is true: *the LLM composes governed metrics / calls deterministic math via a tool it cannot bypass* → 0 hallucinated columns.
- **One grounded LLM brief** — Claude takes the **deterministic** decomposition + significance + dollar figure and writes the exec Decision Brief + one recommended next test. One LLM call, fully grounded (numbers are tool outputs, never model-generated).
- **Honest eval** — `injector.py` perturbs a frozen copy (rate vs volume), **6–10 scenarios** (single/multi mix, single/multi rate, 2 controls), a `scorer.py` , and the **naive "largest-absolute-delta" baseline**. Report the number *and* state plainly what it proves (controlled attribution accuracy) and doesn't (real-world causal accuracy).
- (*Stretch, if a day is free*): one self-critique pass — the LLM re-checks its brief against the numbers. A nod to "the Critic" without the battery.

Deliverable / demo: `python -m helios.run` → governed SQL via the MCP server → decomposition → significance → dollar impact → a Claude-written Decision Brief. `python -m helios.eval` → the honest benchmark vs baseline. + a short case-study writeup with the honest positioning.

The line it earns: *"Built an LLM growth-analysis assistant that diagnoses GA4 funnel movements through a governed semantic layer exposed over MCP — the model composes metrics and calls a deterministic decomposition tool, never writing raw SQL (0 hallucinated columns) — and validated it on a labeled benchmark that beats a naive segment-delta baseline."*

Helios v2 — after the 2 weeks (the *deferred*, not the *flawed*)

Only the things cut for **time**, framed honestly:

- The remaining MCP servers + a **small** multi-agent split (Monitor → Decompose → Diagnose → Narrate) + a **real** self-critique/Critic loop.
- The full labeled benchmark + a **CI eval gate** (GitHub Actions).
- The richer semantic layer (toward the 47 metrics), once each metric earns its keep.
- A real experiment-design module (power/MDE) — clearly labeled "designs experiments; the public data is observational so it cannot run them."
- Cohorts/retention/forecasting **only** with the explicit 3-month-window caveats.
- Memory (seasonality calendar, suppression) **only if** it demonstrably reduces false positives.
- A scheduler / "near-real-time" mode — and an honest note that it's only meaningful on a *live* GA4 export, not the frozen sample.

Explicitly NOT in any version: "diagnoses *why*," "autonomous," causal claims without causal data, or a benchmark whose labels are generated by the system's own algebra. Those are the red-team's fundamental kills; don't reintroduce them.

The 2-week day-by-day

Day	Build	Gate
1	M0 + M1 — repo, GCP/IAM/ADC, <code>dbt_project.yml</code> / <code>profiles.yml</code> / <code>packages.yml</code> , macros, <code>src_ga4.yml</code> , seed	<code>dbt debug</code> green
2	M2 staging + start M3 sessionization	staging tests pass
3	M3 keystone — sessionization + <code>reached_*</code> flags + golden tests first	monotonicity + <code>session_key</code> unique
4	M4 — <code>fct_funnel</code> , <code>fct_daily_funnel</code> , <code>fct_orders</code> , dims + tests	<code>dbt build</code> green; reconciles to the cent ; channels = 10
5	Trimmed ~12-metric semantic YAML + <code>decompose_change</code> + significance + golden decomposition test	unit test passes (<code>mix=-0.0018,...</code>)
6	<code>diagnose.py</code> (movement → decompose → templated brief)	MVP demo runs

Day	Build	Gate
7	Buffer / polish / MVP README + case-study draft	Helios MVP shipped
8	One MCP server (semantic <i>or</i> stats) + connect to Claude Code	grounded query/decompose round-trips
9	The grounded LLM Decision Brief + recommended-next-test	brief generated from tool outputs only
10	<code>injector.py</code> + 6–10 scenarios + <code>scorer.py</code> + naive baseline	eval runs end-to-end
11	Run eval, record the honest number + caveats; (stretch: self-critique pass)	accuracy vs baseline reported
12	Integrate (<code>helios.run</code> one-command) + polish	full loop runs in one command
13	Case study / demo recording / honest positioning + README	writeup done
14	Buffer + interview-story prep (trim <code>INTERVIEW_GUIDE.md</code> to <i>what you actually built</i>)	Helios v1 shipped

Interview ammunition (for the shipped v1)

- **The technique:** walk the mix-vs-rate decomposition with the worked example; explain Simpson's paradox and why a naive "biggest segment delta" gets the *wrong* root cause.
- **The grounding:** the MCP boundary — the model composes governed metrics and calls deterministic math; it physically cannot hand-write SQL, so 0 hallucinated columns.
- **The maturity move (this wins rounds):** *"My eval proves controlled-attribution accuracy against a naive baseline. It does **not** prove real-world causal accuracy — the decomposition tells you where the number moved, not why; the cause (a deploy, a price change) lives outside the data. I scoped to what's honest and provable."* Saying this out loud is a stronger signal than any 85% number.
- **The judgment:** *"The full design was a 7-agent autonomous system. With 2 weeks and one dev I cut it to the governed spine + one grounded LLM step + an honest eval — because the value was in the decomposition and the grounding, not the agent fleet."* That's a senior prioritization story.